
Análise, documentação e manual do projeto **FightZone**

Levantamento de requisitos, modelagem UML, banco de dados, identidade visual e documentação funcional da loja virtual de artigos de boxe.



Documento preparado para apresentação e avaliação do projeto

Versão documental: 2.0 | 7 de outubro de 2026

Aplicação web em PHP, MySQL/MariaDB, HTML, CSS e JavaScript

Sobre este documento

Este livro reúne a análise e a documentação técnica da FightZone e explica o funcionamento do sistema implementado.

A Parte I atende aos tópicos de análise e documentação: levantamento, modelagem UML, banco de dados, dicionário, script SQL, repositório e identidade visual. A Parte II apresenta os perfis, fluxos de uso, regras de negócio, instalação e limitações do sistema, para que a implementação possa ser compreendida e demonstrada.

Precisão do escopo: os requisitos foram organizados a partir da implementação e dos artefatos existentes. O workspace analisado não possui metadados Git. O sistema aceita uma forma de pagamento, mas não cobra; estima frete; e gera comprovante PDF, não nota fiscal eletrônica (NF-e).

Sumário

P1.1 Levantamento de requisitos

P1.2 UML: atores e casos de uso

P1.3 UML: classes e atividades

P1.4 Modelagem do banco de dados

P1.5 Dicionário de dados

P1.6 Script SQL

P1.7 Repositório e organização do projeto

P1.8 Identidade visual

1. Visão geral e tecnologias

2. Perfis e permissões

3. Funcionalidades do cliente

4. Painel e administração

5. Regras de negócio e cálculos

6. Arquitetura e fluxo de dados

7. Segurança e dados pessoais

8. Instalação e operação local

9. Limitações e evolução

10. Roteiro de demonstração

11. Mapa de arquivos e glossário

Índice remissivo

Levantamento de requisitos

A FightZone é uma loja virtual de artigos de boxe com catálogo para clientes e painel de operação para a equipe.

Objetivo e escopo

O sistema apresenta produtos, permite registrar compras de clientes e oferece ferramentas para acompanhar pedidos, catálogo e estoque. O ambiente de instalação documentado é o XAMPP local.

Atores e necessidades

Ator	Objetivo	Recursos oferecidos
Visitante	Conhecer e localizar produtos.	Consultar e pesquisar a vitrine.
Cliente	Comprar produtos e fornecer dados de entrega.	Criar conta, entrar, carrinho, frete estimado, checkout e comprovante.
Vendedor	Acompanhar a operação comercial.	Dashboard, consulta de pedidos e relatórios.
Administrador	Manter catálogo e apoiar a operação de estoque.	Gerir produtos, categorias e entradas; também acessa as funções da equipe.
ViaCEP	Ajudar a localizar um endereço por CEP.	Preenchimento opcional de rua, cidade e UF no checkout.

Requisitos funcionais

ID	Requisito funcional	Ator
RF-01	Exibir produtos ativos, com estoque positivo e categoria ativa.	Visitante/cliente
RF-02	Pesquisar produtos por nome, descrição e categoria.	Visitante/cliente
RF-03	Criar conta de cliente com e-mail válido, senha de ao menos 8 caracteres e confirmação.	Visitante
RF-04	Autenticar cliente e equipe em áreas separadas.	Cliente/equipe
RF-05	Adicionar itens ao carrinho, alterar quantidades e remover itens.	Cliente
RF-06	Calcular estimativa de frete pelo CEP, peso e dimensões do pacote.	Cliente
RF-07	Consultar opcionalmente o ViaCEP para sugerir dados de endereço.	Cliente/sistema
RF-08	Validar entrega e forma de pagamento escolhida e registrar o pedido.	Cliente
RF-09	Confirmar estoque, gravar os itens e baixar o saldo dentro de transação.	Sistema
RF-10	Disponibilizar comprovante PDF com código de acesso ao pedido.	Cliente

RF-11	Exibir indicadores do painel e permitir consulta de pedidos.	Equipe
RF-12	Consultar relatórios resumidos em intervalo de datas.	Equipe
RF-13	Cadastrar/editar/inativar produtos e aceitar imagem validada; gerar SKU automático.	Administrador
RF-14	Criar, editar, ativar e inativar categorias.	Administrador
RF-15	Registrar entrada de estoque, observação e responsável.	Administrador
RF-16	Consultar histórico das movimentações de estoque.	Equipe

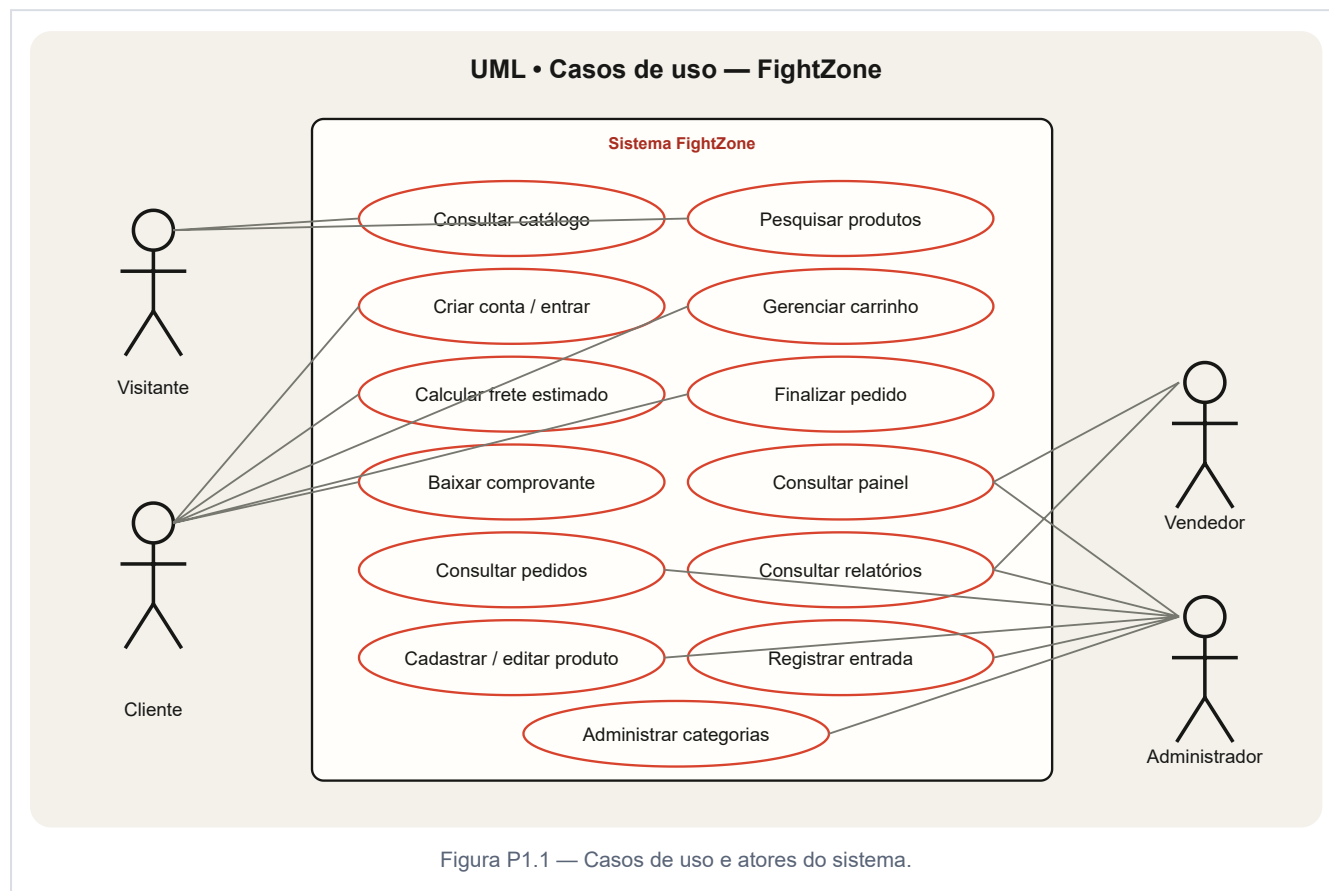
Requisitos não funcionais e restrições

- **RNF-01 — Ambiente:** compatibilidade com o ambiente PHP e MySQL/MariaDB local documentado.
- **RNF-02 — Integridade:** persistência relacional, chaves estrangeiras e transações nas operações críticas de compra/reposição.
- **RNF-03 — Segurança básica:** hash de senha, consultas parametrizadas, escape de saída HTML, token CSRF em POSTs pertinentes e controle de acesso administrativo.
- **RNF-04 — Usabilidade:** interface em português, navegação por áreas e layout responsivo.
- **RNF-05 — Histórico:** inativação lógica para produtos e categorias; snapshot de dados comerciais em pedidos.
- **Restrição — frete:** regra local ilustrativa, não é cotação de transportadora.
- **Restrição — pagamento:** o método é registrado; não existe cobrança externa ou confirmação automática.
- **Restrição — documento fiscal:** o PDF é comprovante e não é NF-e.

Limite metodológico: o workspace não contém atas ou entrevistas com usuários. Este levantamento consolida o escopo observável no código; não afirma a realização de entrevistas formais ou validação externa de requisitos.

UML: atores e casos de uso

O diagrama abaixo delimita o sistema, seus atores e os objetivos principais que cada perfil pode realizar.



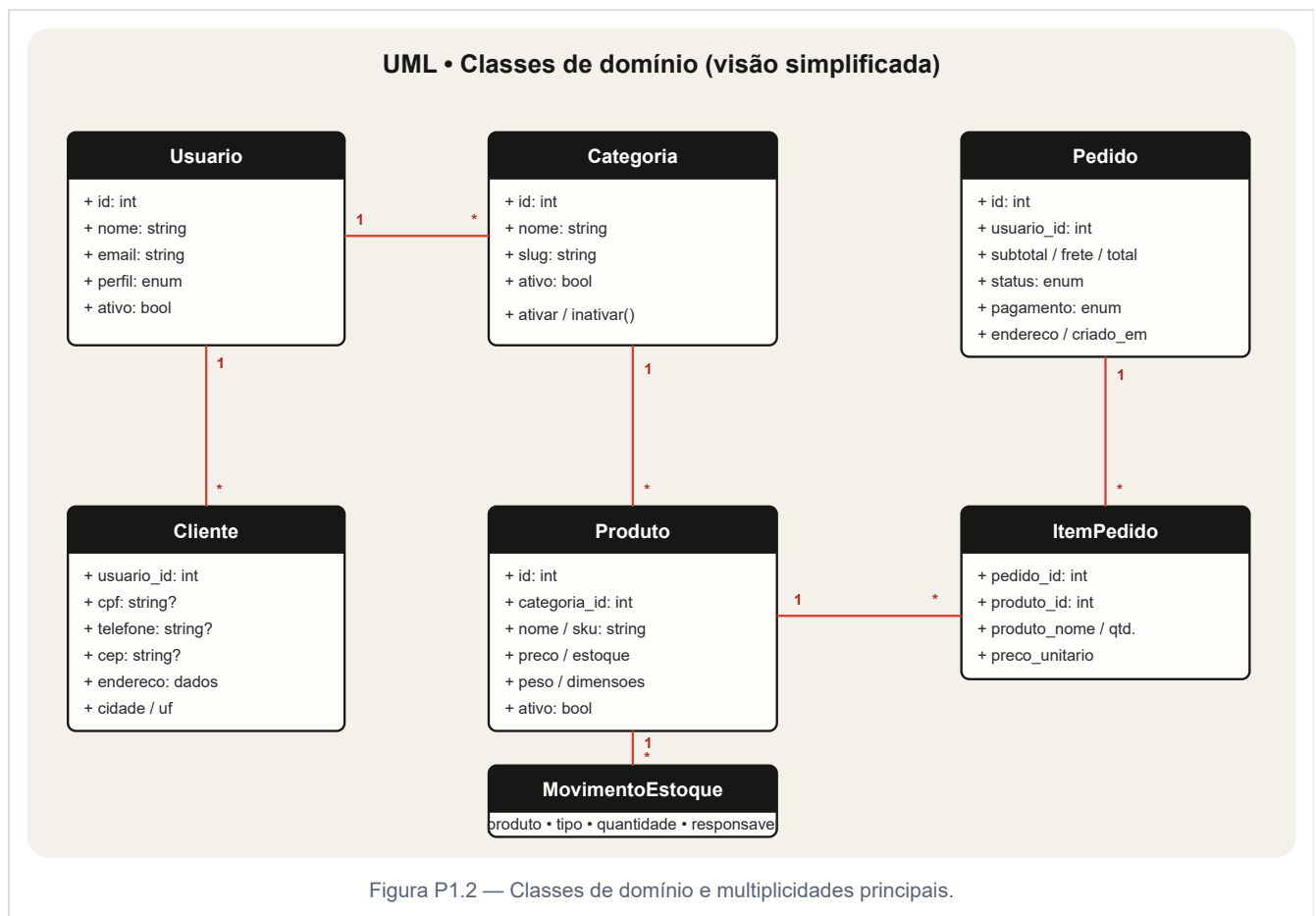
Caso de uso	Ator	Pré-condição	Resultado
Consultar/pesquisar catálogo	Visitante/cliente	Loja disponível.	Produtos disponíveis correspondentes à busca.
Criar conta/entrar	Visitante/cliente	Dados válidos; e-mail livre no cadastro.	Sessão cliente autenticada.
Gerenciar carrinho	Cliente	Autenticado; produto ativo e com estoque.	Carrinho atualizado na sessão.
Estimar frete	Cliente	Carrinho não vazio e CEP de oito dígitos.	Estimativa apresentada.
Finalizar pedido	Cliente	Endereço, método de pagamento e estoque válidos.	Pedido registrado; estoque reduzido.
Baixar comprovante	Cliente com link	ID e código de acesso corretos.	Arquivo PDF do pedido.
Consultar operação	Vendedor/admin	Sessão de equipe ativa.	Painel, pedidos e relatórios.
Manter catálogo/reposição	Administrador	Perfil admin.	Produtos, categorias e estoque atualizados.

Associações sintetizam a navegação e as regras observadas nos controllers; os casos de uso não implicam cobrança financeira nem emissão fiscal.

UML: classes e atividades

Diagrama de classes do domínio

O diagrama de classes resume as entidades persistidas e suas relações principais. É uma visão de domínio, não um inventário de todas as classes PHP. Os atributos completos estão no dicionário de dados.



- **Usuario–Cliente:** uma conta pode ter zero ou um perfil de cliente; usuários de equipe não precisam desse perfil.
- **Categoria–Produto:** cada produto referencia uma categoria; uma categoria pode classificar vários produtos.
- **Usuario–Pedido:** cada pedido pertence a uma conta, que pode ter vários pedidos.
- **Pedido–ItemPedido:** pedido composto por itens; cada item pertence a um pedido.
- **Produto–ItemPedido:** produto pode participar de vários itens; item preserva nome e preço da compra.
- **Produto–MovimentoEstoque:** o produto pode ter vários movimentos; movimento pode guardar o usuário responsável.

Diagrama de atividades: compra

As decisões representam autenticação, validação dos dados de checkout e disponibilidade do estoque. Em caso de falha, a aplicação informa o erro e não confirma a compra.

UML • Atividades — Finalizar uma compra

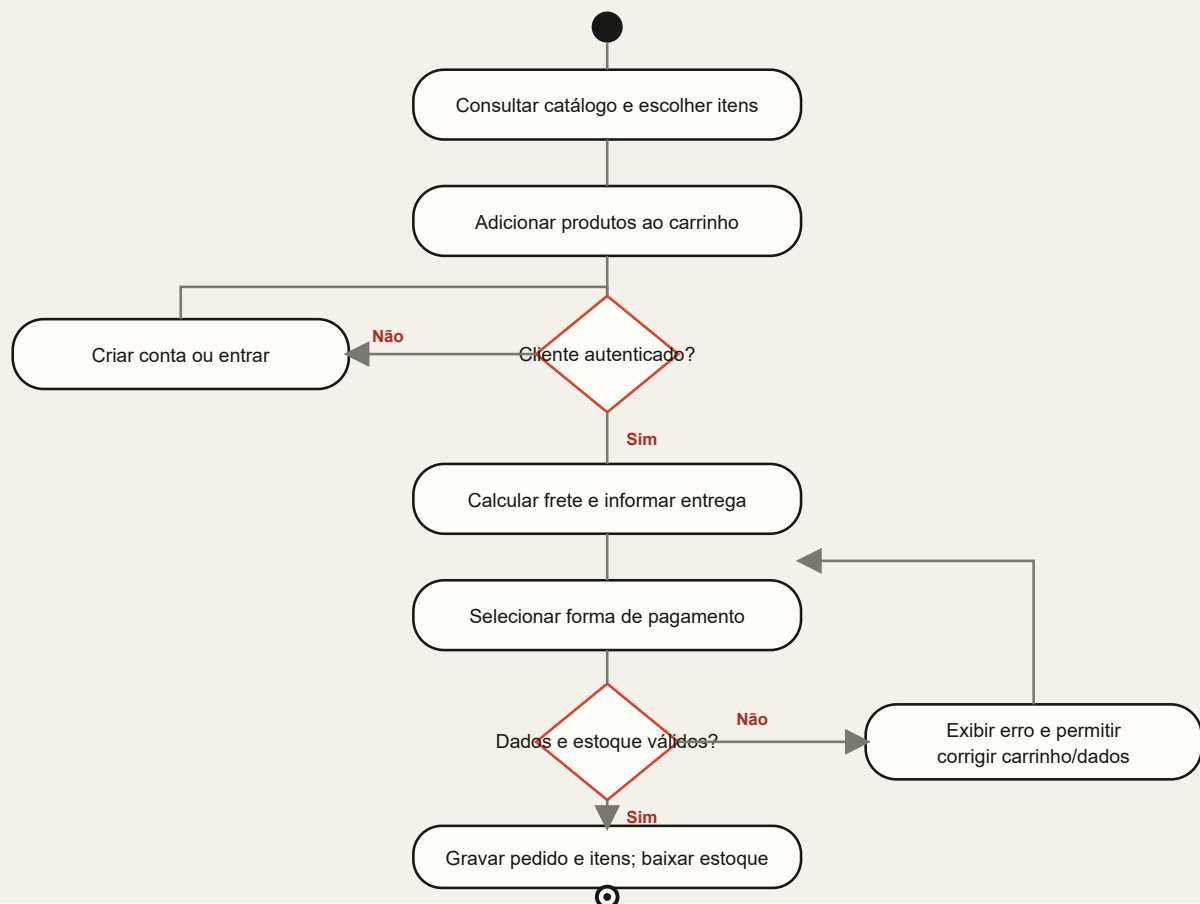


Figura P1.3 — Fluxo de atividades de compra e validação.

Modelagem do banco de dados

O banco relacional `fightzone` separa usuários, perfis de clientes, categorias, produtos, pedidos, itens e histórico de estoque. Chaves primárias identificam registros; FKs ligam dados dependentes; índices únicos impedem algumas duplicidades.

Modelo conceitual e cardinalidades

Relação	Cardinalidade	Regra do domínio
usuarios — clientes	1 para 0..1	Uma conta tem no máximo um perfil de cliente.
categorias — produtos	1 para 0..N	Produto pertence a uma categoria; categoria agrupa produtos.
usuarios — pedidos	1 para 0..N	Cliente pode registrar vários pedidos.
pedidos — itens_pedido	1 para 1..N no fluxo concluído	Um pedido concluído contém suas linhas de produto.
produtos — itens_pedido	1 para 0..N	Produto pode aparecer em pedidos diferentes.
produtos — movimentos_estoque	1 para 0..N	Entrada/saída é registrada por produto.
usuarios — movimentos_estoque	1 para 0..N; movimento pode ser sem usuário	Movimento manual identifica responsável; saídas automáticas podem ser nulas.
pedidos — notas_fiscais	1 para 0..1 por restrição de unicidade	Registro interno; não prova emissão de nota fiscal autorizada.

Modelo relacional, normalização e integridade

A tabela `itens_pedido` resolve a relação de vários produtos em cada pedido e mantém preço/nome como snapshot. Dados de contato e entrega também são copiados para o pedido para preservar o contexto da compra. Valores financeiros usam `DECIMAL`; exclusões de produto e categoria são substituídas por inativação.

As tabelas usam InnoDB e codificação UTF-8. O script declara FKs para as relações principais, índice único para e-mail, slug/SKU e número de registro fiscal interno, além de índices auxiliares em colunas de relacionamento.

Dicionário de dados

NN = obrigatório (*NOT NULL*); AI = auto incremento; — = sem valor padrão explícito. Campos introduzidos por `ALTER TABLE` são listados na estrutura final.

usuarios

Campo	Tipo	Nulo/padrão	Chave e significado
id	INT	NN, AI	PK, identificador.
nome	VARCHAR(100)	NN	Nome de exibição.
email	VARCHAR(150)	NN	Único, login e contato.
senha	VARCHAR(255)	NN	Hash de senha.
perfil	ENUM(admin,vendedor,cliente)	NN, cliente	Papel da conta.
ativo	TINYINT(1)	NN, 1	Conta habilitada.
created_at	DATETIME	NN	Criação da conta.

categorias

Campo	Tipo	Nulo/padrão	Chave e significado
id	INT	NN, AI	PK.
nome	VARCHAR(80)	NN	Nome apresentado.
slug	VARCHAR(100)	NN	Único, identificador textual.
ativo	TINYINT(1)	NN, 1	Disponibilidade.

produtos

Campo	Tipo	Nulo/padrão	Chave e significado
id	INT	NN, AI	PK.
categoria_id	INT	NN	FK categorias.id.
nome	VARCHAR(150)	NN	Nome comercial.
slug	VARCHAR(170)	NN	Único; usado pela aplicação como SKU.
marca	VARCHAR(80)	NN	Marca.
descricao	TEXT	NULL	Descrição.

preco	DECIMAL(10,2)	NN	Preço unitário.
peso	DECIMAL(7,2)	NN, 0.50	Peso por unidade, kg.
altura	DECIMAL(7,2)	NN, 10	Pacote, cm.
largura	DECIMAL(7,2)	NN, 10	Pacote, cm.
comprimento	DECIMAL(7,2)	NN, 10	Pacote, cm.
estoque	INT	NN, 0	Saldo em unidades.
imagem	VARCHAR(255)	NULL	Caminho relativo da imagem.
ativo	TINYINT(1)	NN, 1	Disponibilidade.
created_at	DATETIME	NN	Criação do registro.

clientes

Campo	Tipo	Nulo/padrão	Chave e significado
id	INT	NN, AI	PK.
usuario_id	INT	NN	FK usuarios.id; único, perfil por conta.
nome	VARCHAR(100)	NN	Nome do cliente.
cpf	VARCHAR(14)	NULL	Documento opcional.
telefone	VARCHAR(20)	NULL	Telefone.
cep	VARCHAR(9)	NULL	CEP.
logradouro	VARCHAR(150)	NULL	Rua/avenida.
numero	VARCHAR(20)	NULL	Número.
complemento	VARCHAR(100)	NULL	Complemento.
bairro	VARCHAR(100)	NULL	Bairro.
cidade	VARCHAR(100)	NULL	Cidade.
uf	CHAR(2)	NULL	Unidade federativa.

pedidos

Campo	Tipo	Nulo/padrão	Chave e significado
id	INT	NN, AI	PK.
usuario_id	INT	NN	FK usuarios.id, proprietário.
subtotal	DECIMAL(10,2)	NN	Soma dos produtos.

frete	DECIMAL(10,2)	NN	Frete estimado.
total	DECIMAL(10,2)	NN	Subtotal + frete.
status	ENUM(aguardando,pago,separacao,enviado,entregue,cancelado)	NN, aguardando	Estado do pedido.
pagamento	ENUM(pix,cartao,boleto)	NN	Forma escolhida.
cep_entrega	VARCHAR(9)	NULL	CEP destino.
endereco_entrega	TEXT	NULL	Endereço da entrega.
created_at	DATETIME	NN	Data do pedido.
codigo_acesso	CHAR(48)	NULL	Código de autorização do comprovante.
cliente_nome	VARCHAR(160)	NULL	Snapshot de nome.
cliente_email	VARCHAR(190)	NULL	Snapshot de e-mail.
cliente_documento	VARCHAR(20)	NULL	Documento informado.
cliente_telefone	VARCHAR(30)	NULL	Telefone.
status_pagamento	VARCHAR(30)	NULL	Estado textual de pagamento; não há confirmação integrada.

itens_pedido

Campo	Tipo	Nulo/padrão	Chave e significado
id	INT	NN, AI	PK.
pedido_id	INT	NN	FK pedidos.id.
produto_id	INT	NN	FK produtos.id.
quantidade	INT	NN	Unidades compradas.
preco_unitario	DECIMAL(10,2)	NN	Preço congelado na compra.
produto_nome	VARCHAR(150)	NULL	Nome congelado na compra.

movimentos_estoque

Campo	Tipo	Nulo/padrão	Chave e significado
id	INT	NN, AI	PK.
produto_id	INT	NN	FK produtos.id.
tipo	ENUM(entrada,saida,ajuste)	NN	Natureza do movimento.

quantidade	INT	NN	Unidades movimentadas.
observacao	VARCHAR(255)	NULL	Motivo/referência.
usuario_id	INT	NULL	FK usuarios.id, responsável opcional.
created_at	DATETIME	NN	Data do movimento.

notas_fiscais

Campo	Tipo	Nulo/padrão	Chave e significado
id	INT	NN, AI	PK.
pedido_id	INT	NN	FK pedidos.id, único.
numero	VARCHAR(50)	NN	Único, número interno.
valor_total	DECIMAL(10,2)	NN	Total registrado.
created_at	DATETIME	NN	Data do registro.

Nota: a tabela `notas_fiscais` é apenas registro interno no projeto. Campos chamados de nota fiscal não significam NF-e autorizada. O schema não declara CHECK para preço, saldo ou dimensões; a aplicação valida boa parte dessas regras em PHP.

Script SQL

O artefato autoritativo para criação/evolução do schema é `database/fightzone.sql`. Ele cria e seleciona a base `fightzone`, declara as tabelas InnoDB em UTF-8, PKs, FKs e índices e inclui dados iniciais de categorias.

Conteúdo do script

1. Criação do banco com conjunto de caracteres `utf8mb4` e collation Unicode.
2. Criação de `usuarios`, `categorias`, `produtos`, `clientes`, `pedidos`, `itens_pedido`, `movimentos_estoque` e `notas_fiscais`.
3. Definição de chaves únicas para e-mail, slugs e número da nota interna e FKs para as relações centrais.
4. Inclusão de colunas adicionais de pedido e snapshot de produto por comandos `ALTER TABLE ... ADD COLUMN IF NOT EXISTS`.
5. Carga repetível das categorias iniciais com `INSERT IGNORE`.

Execução e cuidados

1. Inicie o MySQL/MariaDB no XAMPP e abra phpMyAdmin.
2. Importe `database/fightzone.sql`.
3. Confirme a existência das tabelas e execute `check.php`.

As operações de criação e carga inicial são concebidas para tolerar reexecução; o suporte à sintaxe `ADD COLUMN IF NOT EXISTS` depende da versão do servidor MySQL/MariaDB. Faça backup antes de atualizar uma base com dados importantes.

Para evitar divergência, o manual resume, mas não duplica o script SQL integral. Na entrega, mantenha `database/fightzone.sql` junto do código como fonte autoritativa do schema.

Repositório e organização do projeto

O projeto está organizado em pastas por responsabilidade. A estrutura facilita relacionar controllers, models e views com os casos de uso, mantendo o banco e os artefatos de documentação junto da aplicação.

Local	Conteúdo/finalidade
index.php	Ponto de entrada, inicialização de sessão e roteamento controller/action.
config/	Configuração de infraestrutura; conexão PDO com MySQL/MariaDB.
controllers/	Coordenação de autenticação, loja, produtos, categorias, pedidos, estoque e relatórios.
models/	Consultas e persistência das entidades do domínio.
views/	Páginas HTML/PHP da loja, checkout e painel.
services/	Serviços isolados, como geração de comprovante PDF.
database/	Script SQL de schema e categorias iniciais.
public/assets/	CSS e ilustrações públicas do catálogo.
uploads/	Destino de imagens enviadas pelo administrador.
docs/	Manual, diagramas UML e guias visuais/fluxogramas.
README.md	Instruções resumidas de configuração e uso.

Controle de versão e evidência de entrega

No workspace examinado não foi encontrado diretório `.git`. Assim, é possível documentar a estrutura do projeto, mas não afirmar que esta cópia está ligada a um remoto, que possui histórico de commits, branch publicado ou URL pública. Caso a avaliação exija repositório Git, a equipe deve vincular o projeto ao repositório autorizado e entregar a URL/branch efetivamente existentes.

Para complementar: versionar o código, este manual, os diagramas e o SQL; manter credenciais e dados pessoais fora do Git; registrar alterações em commits descritivos e incluir na apresentação a URL real do remoto. Não invente uma URL ou histórico que não exista.

Identidade visual

A marca FightZone comunica esporte de combate, energia e desempenho usando contraste forte, acento vermelho e títulos condensados, sem sacrificar a legibilidade dos formulários e tabelas.

Assinatura e linguagem

- **Nome da marca:** FIGHTZONE, reforçado por contraste entre as palavras “FIGHT” e “ZONE”.
- **Monograma:** “FZ” em selo vermelho inclinado, construído no HTML/CSS.
- **Descritor de loja:** “BOXE • TREINO • PERFORMANCE”; no painel, “PAINEL DE GESTÃO”.
- **Tom de comunicação:** curto, esportivo, motivador e claro nas etapas de compra.

Paleta e tipografia

Elemento	Token/valor	Uso
Carvão	#171816	Marca, cabeçalhos, painel e texto principal.
Papel quente	#f4f1eb	Fundo das páginas e ilustrações.
Branco quente	#fffffa	Cartões, tabelas e áreas de formulário.
Vermelho	#d8432e	Monograma, ação principal e destaques.
Vermelho escuro	#a92d20	Estados de interação e ênfase.
Cinza auxiliar	#777870	Legendas e texto secundário.
Divisória	#dedbd4	Bordas leves e linhas.
Títulos	Barlow Condensed; fallback Impact/sans-serif	Títulos esportivos, fortes e condensados.
Texto de interface	DM Sans; fallback Arial/sans-serif	Leitura, navegação, formulários e dados.

Aplicação consistente

A loja usa cards de produto, chamada principal escura e botões vermelhos. O painel usa menu lateral escuro, cartões de indicadores e ações rápidas. O CSS da loja importa Barlow Condensed e DM Sans do Google Fonts; em indisponibilidade de rede, entram fontes alternativas. O monograma é desenhado pela interface; não há nesta versão um arquivo vetorial separado de logotipo.

As referências encontram-se em `public/assets/css/fightzone.css`, `public/assets/css/dashboard.css`, `public/assets/css/produtos.css` e nas ilustrações de `docs/`.

Visão geral e tecnologias

FightZone é uma loja de equipamentos de boxe e painel de operação construídos como aplicação PHP com banco MySQL/MariaDB.

Tecnologia	Função no projeto
PHP	Roteamento, regras de negócio, controllers, models e renderização de páginas.
PDO	Conexão ao banco e execução de consultas parametrizadas.
MySQL/MariaDB	Persistência de contas, catálogo, pedidos e estoque.
HTML/CSS	Estrutura e apresentação de loja e painel.
JavaScript	Atualização visual de frete e integração opcional de consulta ViaCEP.
XAMPP	Ambiente local de desenvolvimento descrito pelo projeto.

Finalidade

O cliente percorre uma compra simulada de ponta a ponta. A equipe acompanha indicadores e pedidos e, com perfil administrador, mantém o catálogo e registra entradas de estoque. As integrações comerciais reais não fazem parte da implementação atual.

Perfis e permissões

Perfil	Recursos da interface
Visitante	Consultar/pesquisar produtos; abrir entrada/cadastro de cliente.
Cliente	Criar conta, entrar, usar carrinho e finalizar compra.
Vendedor	Consultar dashboard, pedidos e relatórios.
Administrador	Funções da equipe mais produtos, categorias e entradas de estoque.

Autenticação

O cadastro público cria somente usuários do perfil `cliente`. O login interno aceita contas ativas com perfil `admin` ou `vendedor`, e fica separado do login do cliente. A senha é verificada contra hash salvo e o identificador de sessão é regenerado na autenticação.

Limites de autorização observados

Gravação de produto, remoção, reposição e gravação/alteração de categoria verificam perfil administrador. As listagens de pedidos e relatórios verificam autenticação de equipe, sem restrição adicional de perfil no controller. Menus de administração são exibidos condicionalmente. Cadastro inicial de equipe é feito fora do cadastro público e está explicado no README.

Funcionalidades do cliente

Catálogo → Conta → Carrinho → Frete → Entrega e forma de pagamento → Pedido → Comprovante PDF

Conta

Cadastro pede nome, e-mail, senha e confirmação. A aplicação verifica formato/tamanho do e-mail, tamanho mínimo da senha, coincidência e duplicidade de e-mail. Login confere perfil cliente, estado ativo e senha. Minha conta exibe nome e e-mail; sair limpa a sessão de compra e seus dados temporários.

Catálogo e pesquisa

A vitrine lista apenas produto ativo, com estoque maior que zero, e categoria ativa. A pesquisa compara termo com nome, descrição e categoria. Cartões exibem imagem (ou monograma FightZone), nome, descrição, categoria, preço e SKU.

Carrinho

Adicionar requer cliente autenticado. A unidade adicionada é limitada ao estoque atual. O cliente altera quantidades; quantidade zero remove. Ao carregar, a aplicação confirma disponibilidade e ajusta/remover itens indisponíveis.

Entrega e comprovante

O cliente informa CEP, rua, número, complemento, cidade, UF, telefone e documento opcional. ViaCEP pode preencher alguns campos, que devem ser conferidos. A confirmação apresenta o número do pedido e o link para PDF com token de acesso.

O PDF reúne cliente, entrega, itens, quantidades, preços, subtotal, frete, total e forma de pagamento. Ele é comprovante/resumo e avisa que não substitui NF-e.

Painel e administração

Dashboard

Exibe pedidos não cancelados no mês, soma dos totais dos pedidos pagos criados no mês, pedidos aguardando, produtos ativos com cinco unidades ou menos e quantidade de produtos ativos. Os indicadores são calculados a partir do banco.

Pedidos e relatórios

Pedidos mostra ID, data, cliente, e-mail, forma escolhida, status e total. É uma consulta; a tela não permite transicionar status. Relatórios filtram datas e mostram quantidade, valor total de todos os pedidos do período, contagem de produtos ativos, total de unidades em estoque e lista dos pedidos.

Produtos e estoque

Administrador cadastra/edita nome, descrição, categoria, marca, preço, estoque, peso, dimensões e imagem opcional JPG/PNG/WebP de até 5 MB. SKU de novos itens é automático no padrão FZ-000001 ; permanece estável ao editar. Produtos removidos ficam inativos. A edição do cadastro pode alterar diretamente o saldo, sem criar movimento; reposição deve usar a função de entrada.

Entradas de estoque

Administrador escolhe produto ativo, registra quantidade positiva e observação opcional. O sistema registra movimento, aumenta o saldo e vincula responsável na mesma transação. Histórico lista data, produto, tipo, quantidade, responsável e observação. A finalização de pedido registra saída automática.

Categorias

Administrador cria, edita, ativa e inativa. Não se apaga fisicamente; categoria inativa não é oferecida para cadastro/vitrine.

Regras de negócio e cálculos

Frete estimado

O CEP tem oito dígitos; seus dois primeiros escolhem valor base. Para cada unidade usa-se o maior entre peso real e peso cúbico, que é $\text{altura} \times \text{largura} \times \text{comprimento} \div 6000$, com dimensões em centímetros e resultado aproximado em quilogramas. Soma-se o peso de todas as unidades. Até 1 kg vale a base; acima disso soma-se R\$ 3,50 por quilograma excedente; o resultado é arredondado a centavos.

Prefixo CEP	Base
00–19	R\$ 16,90
20–39	R\$ 19,90
40–65	R\$ 26,90
66–69	R\$ 34,90
70–79	R\$ 24,90
80–99	R\$ 21,90

Estimativa pedagógica local: não é preço oficial, cotação, etiqueta nem contratação de transportadora.

Pedido, pagamento e estoque

O pedido novo recebe status `aguardando` e pagamento pendente. PIX, cartão ou boleto são apenas escolhas armazenadas; nenhum dinheiro é processado. Antes de criar o pedido o servidor valida novamente o saldo sob bloqueio transacional. Criação do pedido, itens, baixa do estoque e movimentos ocorrem juntos; falha cancela a transação. Estoque é baixado mesmo com pagamento pendente.

SKU

O produto novo recebe prefixo `FZ-` e sequência de seis dígitos, como `FZ-000123`, baseada no ID do registro e verificada contra colisões. A aplicação usa a coluna SQL `slug` como SKU. Produtos antigos conservam código anterior.

Estados do pedido

O schema prevê aguardando, pago, separação, enviado, entregue e cancelado. No painel atual a listagem é somente leitura; não há tela operacional para avançar esses estados.

Arquitetura e fluxo de dados

Camada	Responsabilidade	Exemplos
Entrada/roteador	Inicializar sessão e encaminhar controller/action.	<code>index.php</code>
Controller	Coordenar acesso, validação, modelo e resposta.	<code>controllers/LojaController.php</code>
Model	Consultar e persistir dados.	<code>models/Compra.php</code> , <code>produto.php</code>
View	Renderizar HTML e formulários.	<code>views/loja.php</code>
Service	Gerar saída especializada.	<code>services/InvoicePdf.php</code>

Roteamento

O padrão de URL é `index.php?controller=...&action=...`. Sem parâmetros, abre `loja/catalogo`. O roteador verifica controller e método. Áreas principais: auth, cliente, loja, produto, entrada, venda, relatorio e categoria.

Transação da compra

1. Catálogo consulta produtos disponíveis.
2. Carrinho e tokens ficam na sessão.
3. Checkout valida dados e calcula frete novamente no servidor.
4. Compra bloqueia registros de produto e confirma estoque suficiente.
5. Perfil de entrega, pedido, snapshots, itens, baixa e movimentos são persistidos em transação.
6. Confirmação usa ID e token; o serviço produz o arquivo PDF.

Arquivos CSS ficam em `public/assets/css/`; imagens carregadas pelo painel ficam em `uploads/`. Arquivos auxiliares isolados de pagamento/funcionários não estão ligados ao roteamento principal e não devem ser apresentados como módulos acessíveis completos.

Segurança e dados pessoais

- Senhas com `password_hash` e verificadas por `password_verify`.
- Consultas parametrizadas PDO para dados variáveis.
- Escape HTML com `htmlspecialchars` nas vistas principais.
- Tokens CSRF para POSTs das áreas de cliente/loja/estoque que os implementam.
- Regeneração de sessão na autenticação.
- Verificação de perfil admin nas gravações administrativas.
- Upload limitado a 5 MB, com MIME verificado e extensão allowlist.
- Token aleatório de 48 caracteres hexadecimais para acessar o comprovante.

O sistema guarda nome, e-mail, documento opcional, telefone e endereço. O banco XAMPP de exemplo usa root sem senha e serve somente para ambiente local. Operação pública exige configuração de credenciais restritas, HTTPS, backups, controle de acesso, logs, retenção e revisão apropriada de segurança e proteção de dados.

Instalação e operação local

1. Coloque o projeto em `C:\xampp\htdocs\fightzone`.
2. Inicie Apache e MySQL no XAMPP.
3. Importe `database/fightzone.sql` pelo phpMyAdmin em `http://localhost/phpmyadmin`.
4. Confira `config/db.php`: `localhost`, usuário `root`, senha vazia e banco `fightzone` por padrão.
5. Abra a loja em `http://localhost/fightzone/`.

O login da equipe é `http://localhost/fightzone/index.php?controller=auth&action=form`. Conta administrativa inicial deve ser criada/promovida conforme o README, salvando hash e nunca senha em texto puro. `check.php` verifica tabelas principais. `atualizar_senha.php` funciona pela CLI e recusa acesso HTTP.

Limitações e possibilidades de evolução

Tema	Estado atual	Possível evolução
Pagamento	Forma é armazenada; cobrança não é processada.	Provedor, callbacks assinados e conciliação.
Frete	Cálculo ilustrativo local.	API oficial, cotação e rastreio.
Fiscal	Comprovante PDF e registro interno; sem autorização SEFAZ.	Emissor autorizado, XML e DANFE válidos.
Status	Listagem não altera estado do pedido.	Transições autorizadas, histórico e notificações.
Cliente	Sem área de histórico completo de compras.	Histórico autenticado e acompanhamento.
Estoque	Reposições e baixa automática; edição de saldo pode ser direta.	Ajustes sempre auditáveis e inventário.
Produção	Ambiente descrito é local.	HTTPS, backups, monitoramento e implantação.

Schema enumera estados adicionais e existe tabela `notas_fiscais`, mas isso não equivale a funções completas de transição de pedido ou emissão fiscal eletrônica. Os controllers auxiliares de pagamentos e funcionários não são roteados pelo ponto de entrada atual e o schema principal não declara as tabelas desses módulos.

Roteiro de demonstração

1. Apresente identidade visual e pesquisa na vitrine.
2. Crie conta cliente e explique validações e perfis separados.
3. Adicione item, altere quantidade e mostre remoção por quantidade zero.
4. Calcule CEP e explique que frete é estimado.
5. Mostre checkout, seleção de forma e explique que não há cobrança real.
6. Finalize pedido, consulte confirmação e comprovante; explique baixa do estoque.
7. Entre como equipe e apresente dashboard, pedidos e relatório.
8. Como admin, mostre SKU automático, categoria e reposição.
9. Conclua com schema, UML, requisitos e limites/futuras integrações.

Use dados de demonstração fictícios. Faça backup do banco antes de apresentações importantes e evite projetar dados pessoais reais.

Mapa de arquivos e glossário

Local	Conteúdo
index.php	Roteador central.
config/db.php	Conexão de banco por PDO.
controllers/	Coordenação de ações.
models/	Consultas e persistência.
views/	Interfaces PHP/HTML.
services/InvoicePdf.php	Comprovante de pedido.
database/fightzone.sql	Schema e carga inicial.
public/assets/css/	Folhas de estilo.
public/assets/products/	Ilustrações de produtos.
uploads/	Imagens do catálogo.
docs/	Manual e diagramas/guias SVG.

Termo	Definição
CSRF	Falsificação de requisição entre sites; token valida formulários protegidos.
Checkout	Etapa de entrega e escolha de pagamento.
FK	Chave estrangeira que referencia outra tabela.
MVC	Model, View e Controller: dados, apresentação e coordenação.
PDO	Interface PHP para banco de dados.
PK	Chave primária que identifica cada registro.
Snapshot	Cópia histórica de nome/preço/dados em pedido.
SKU	Código único de identificação de produto.
Transação	Operações confirmadas juntas ou revertidas juntas.
ViaCEP	Serviço externo de busca de endereço por CEP.

Índice

Administrador — 2, 4

Banco de dados — P1.4, P1.5, P1.6

Carrinho — P1.2, 3

Categoria — P1.5, 4

Checkout — P1.3, 3

Cliente — P1.1, 2, 3

Comprovante PDF — 3, 9

CSRF — 7, 11

Dicionário de dados — P1.5

Estoque — 4, 5

Frete — 3, 5

Identidade visual — P1.8

MySQL — P1.4, P1.6

Nota fiscal — P1.5, 9

Pedido — P1.2, P1.4, 3, 5

Repositório — P1.7

Requisitos — P1.1

Script SQL — P1.6

SKU — P1.5, 5

UML — P1.2, P1.3

Vendedor — 2, 4

ViaCEP — P1.1, 3

Fim do Manual do Projeto FightZone. O README e os artefatos SQL/UML da pasta docs acompanham este documento.